EnergyWatch AI

DOCUMENTATION TECHNIQUE

Documentation technique

Architecture, agents, observabilité, sécurité, tests, déploiement et correspondance avec le cours de la plateforme EnergyWatch AI.

Projet	**EnergyWatch AI, plateforme multi-agents (LangGraph)**
Secteur	**Énergie**
Cadre	**Mini-projet S5 · Agent en production**
Version	**1.2.0**
Date	**30 octobre 2026**
Dépôt	**github.com/bouincha/Energy-Watch-AI**

Sommaire

1. Présentation du projet
2. Architecture
3. Observabilité et fiabilité
4. Captures du tableau de bord
5. Sécurité et contrôle humain
6. Tests et évaluation
7. Déploiement
8. Correspondance avec le cours
9. Limites et perspectives

Présentation du projet

Le problème

Une entreprise exploite quatre sites très différents au Maroc. Chacun gaspille de l'énergie à sa manière, et personne n'a le temps de lire 16 sous-compteurs toutes les heures :

Site	Type	Gaspillage typique
Siège de Casablanca (`HQ-CASA`)	Bureaux, 12 000 m²	Climatisation et éclairage allumés la nuit ou le week-end
Usine d'assemblage de Kénitra (`PLANT-KEN`)	Usine + panneaux solaires en toiture	Fuites d'air comprimé
Datacenter de Rabat (`DC-RABAT`)	Centre de données	Dégradation du refroidissement (dérive du PUE)
Clinique de Marrakech (`CLINIC-MRK`)	Santé, 24 h/24	Pannes de groupe froid

Le jeu de données simulé (30 jours × 16 compteurs, au pas horaire) contient **5 anomalies réalistes injectées volontairement**, représentant environ **24 000 MAD** de coût évitable, que le système doit trouver, expliquer et corriger.

La solution : EnergyWatch AI

- **Répond aux questions en langage naturel**, par exemple « Y a-t-il des anomalies ce mois-ci ? » ou « Comment réduire le coût du refroidissement du datacenter ? ».
- **Détecte les anomalies** par une méthode statistique (z-score robuste par rapport à une référence par heure et par type de jour) et les classe : fonctionnement de nuit, de week-end, hausse continue, dérive, pic. Les 5 anomalies injectées sont trouvées, sans aucun faux positif, sur 210 combinaisons de données.
- **Recommande des actions fondées sur une base documentaire** (RAG sur 7 guides d'efficacité énergétique), avec citation des sources et estimation des économies mensuelles.
- **Garde l'humain dans la boucle** : les agents ne font que *proposer* des actions. Le risque faible est approuvé automatiquement ; le risque moyen ou élevé attend un opérateur. Chaque décision est inscrite dans une piste d'audit non modifiable.
- **Applique des garde-fous** (OWASP Top 10 LLM) : injection de prompt, fuite de secrets, questions hors sujet, limitation du débit, outils en lecture seule.
- **Se surveille lui-même** : chaque exécution reçoit un **correlation ID** qui relie l'en-tête HTTP, les logs JSON, les étapes des agents, les métriques, les incidents et les entrées d'audit.
- **Gère ses incidents** : disjoncteur, mode dégradé, 4 runbooks, incidents ouverts et résolus automatiquement avec mesure du MTTR.
- **Propose un tableau de bord** de 7 pages pensé pour des non-spécialistes : Home, Ask the AI, Problems found, Approvals, Monitoring, Quality et Governance, avec un bouton « What is this page? » sur chaque page.

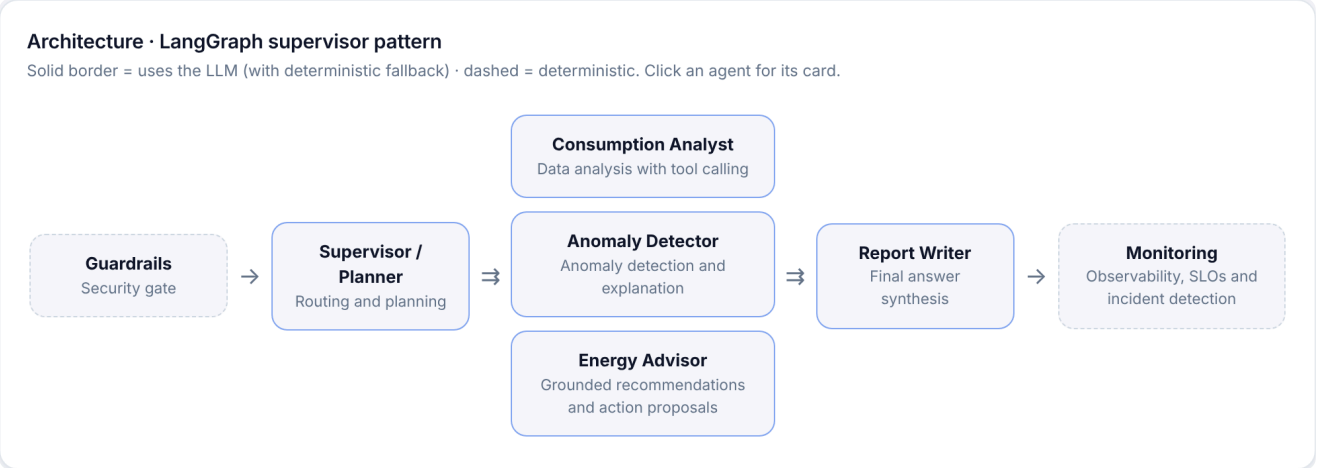
Critères de validation et preuves

Critère	Mise en œuvre	Preuve
Observabilité : correlation ID sur chaque exécution	`app/core/observability.py`, enveloppe `traced()` et `monitor_node` dans `app/agents/nodes.py`	En-tête `X-Correlation-ID`, logs JSON, page Monitoring, test `test_every_run_has_a_correlation_id_on_every_span`
Tests : suite exécutée avec succès	70 tests (unitaires, intégration, bout en bout, seuils d'évaluation), 95 % de couverture	GitHub Actions et page Quality
Gouvernance : Agent Card complète et à jour	`app/governance/agent_cards.json` + document Agent Card	Test de CI qui échoue si une fiche est incomplète ou désynchronisée
Conteneurisation et déploiement	`Dockerfile` multi-étapes, utilisateur non-root, healthcheck ; lancement en ligne avec Cloudflare Tunnel	Job CI « Docker build & smoke test », `scripts/smoke_test.py`
Incident : runbook appliqué, incident géré	`app/core/incidents.py` (RB-001 à RB-004), contrôleur de chaos	Runbook et rapport d'incident INC-0001
Documentation technique + démo vidéo	Le présent document et le dossier `docs/`	Script de la vidéo dans `docs/DEMO_VIDEO_SCRIPT.md`

Architecture

Vue d'ensemble

EnergyWatch AI est un **système multi-agents** construit avec **LangGraph** et exposé par un service **FastAPI** qui sert aussi le tableau de bord. Une requête correspond à une *exécution* du graphe, et chaque exécution est identifiée par un **correlation ID**.



Architecture du système multi-agents LangGraph (capture de la page Governance)

Modèle superviseur / planificateur avec répartiteur. Le superviseur produit un plan (liste ordonnée de spécialistes). Après chaque nœud, l'arête conditionnelle `next_step()` choisit le premier agent prévu qui n'a pas encore tourné, ou passe au rédacteur du rapport. Le graphe est défini dans le code (`app/agents/graph.py`) : le LLM ne peut choisir que parmi les agents autorisés, car sa sortie est validée et assainie.

Les 7 agents

Agent	LLM ?	Responsabilité	Repli
`guardrails`	non	Contrôle injection / hors sujet / longueur avant tout appel au LLM	-
`supervisor`	oui (sortie structurée)	Intention, site, période, plan	Planificateur par mots-clés
`consumption_analyst`	oui (appel d'outils)	Choisit et appelle les outils de données, résume	Choix d'outils par règles + modèle de texte
`anomaly_detector`	oui (explication)	Détection statistique, causes probables	Table des causes
`energy_advisor`	oui (RAG)	Recherche documentaire, recommandations, propositions d'actions	Recommandations prédéfinies + citations
`report_writer`	oui	Réponse finale en Markdown, filtre de sortie	Modèle par sections

Agent	LLM ?	Responsabilité	Repli
`monitor`	non	Vérification des SLO, métriques, incidents	-

État partagé

`EnergyState` (`app/agents/state.py`) est un `TypedDict`. La plupart des clés sont écrites par l'agent qui en est responsable ; quatre clés utilisent des **réducteurs** pour que chaque agent puisse y ajouter : `completed` (agents déjà exécutés), `trace` (un span par agent), `errors` (pannes contenues) et `degraded` (vrai si un agent a utilisé son repli).

Choix de conception

Choix	Justification
Les chiffres viennent d'outils déterministes, jamais du LLM	Évite les kWh et MAD inventés (OWASP LLM09). Le LLM planifie, choisit les outils et rédige.
Chaque agent LLM a un repli déterministe	Le service reste utile pendant une panne du fournisseur (RB-001).
Une passerelle LLM unique (`app/llm/gateway.py`)	Délais, nouvelles tentatives, disjoncteur, chaos, métriques et comptage centralisés.
Fournisseur simulé (mock)	Tests et CI gratuits, rapides et reproductibles ; démo possible hors ligne.
Recherche BM25 en Python pur	Pas de base vectorielle à exploiter pour 29 extraits ; l'interface `search()` peut être remplacée.
Détection statistique (z-score robuste)	Explicable et rapide (≈ 20 ms pour 16 compteurs × 720 h), évaluée sur une vérité terrain.
Validation humaine des actions à risque	Autonomie excessive (OWASP LLM06) ; les sites clinique et datacenter sont critiques.
Télémétrie en mémoire + logs JSON	Simple pour une instance ; les logs sont la trace durable.

Données et détection d'anomalies

`app/data/energy_data.py` simule 4 sites, 16 compteurs et 30 jours de données horaires avec des profils réalistes (occupation, équipes, température extérieure, courbe solaire) et 5 anomalies injectées.

Pour chaque compteur : 1. Référence = médiane par couple *(jour ouvré ?, heure)* sur les 14 premiers jours. 2. Échelle robuste = MAD des résidus d'apprentissage (avec un plancher). 3. Les heures avec z > 3,5 et un écart d'au moins 12 % sont signalées, puis regroupées (trous ≤ 16 h comblés). 4. Filtre anti-bruit : au moins 3 heures ou z ≥ 12, et un coût ≥ 100 MAD. 5. Classement du motif : week-end, dérive / hausse continue, nuit, pic. 6. Chaque événement reçoit les kWh excédentaires, le coût au tarif horaire et une gravité.

Observabilité et fiabilité

Parcours du correlation ID

1. Le middleware `correlation_middleware` (`app/main.py`) lit l'en-tête `X-Correlation-ID` ou en crée un (`ewa-` + 20 caractères hexadécimaux). Une valeur non sûre est remplacée.
2. L'identifiant est stocké dans une `ContextVar` et renvoyé dans l'en-tête de la réponse.
3. `run_agent()` le place dans l'état du graphe et dans le registre des exécutions.
4. `traced()` le rattache à chaque nœud et enregistre un **span** (agent, durée, statut, outils, erreur).
5. Le formateur JSON l'ajoute à **chaque ligne de log** ; incidents, entrées d'audit et actions proposées le conservent aussi.

Ainsi, `docker logs energywatch | grep <correlation-id>` raconte toute l'histoire d'une requête.

Métriques et SLO

- Métriques Prometheus sur `/metrics` : exécutions par statut, latence par agent et de bout en bout, appels LLM (succès, échec, court-circuit), blocages des guardrails, incidents, actions proposées.
- SLO : latence de bout en bout ≤ 8 000 ms, taux d'erreur ou de dégradation ≤ 20 %.
- La page **Monitoring** affiche les percentiles p50 / p95, le taux d'erreur, l'état du disjoncteur, l'explorateur d'exécutions (cascade par agent) et les incidents.

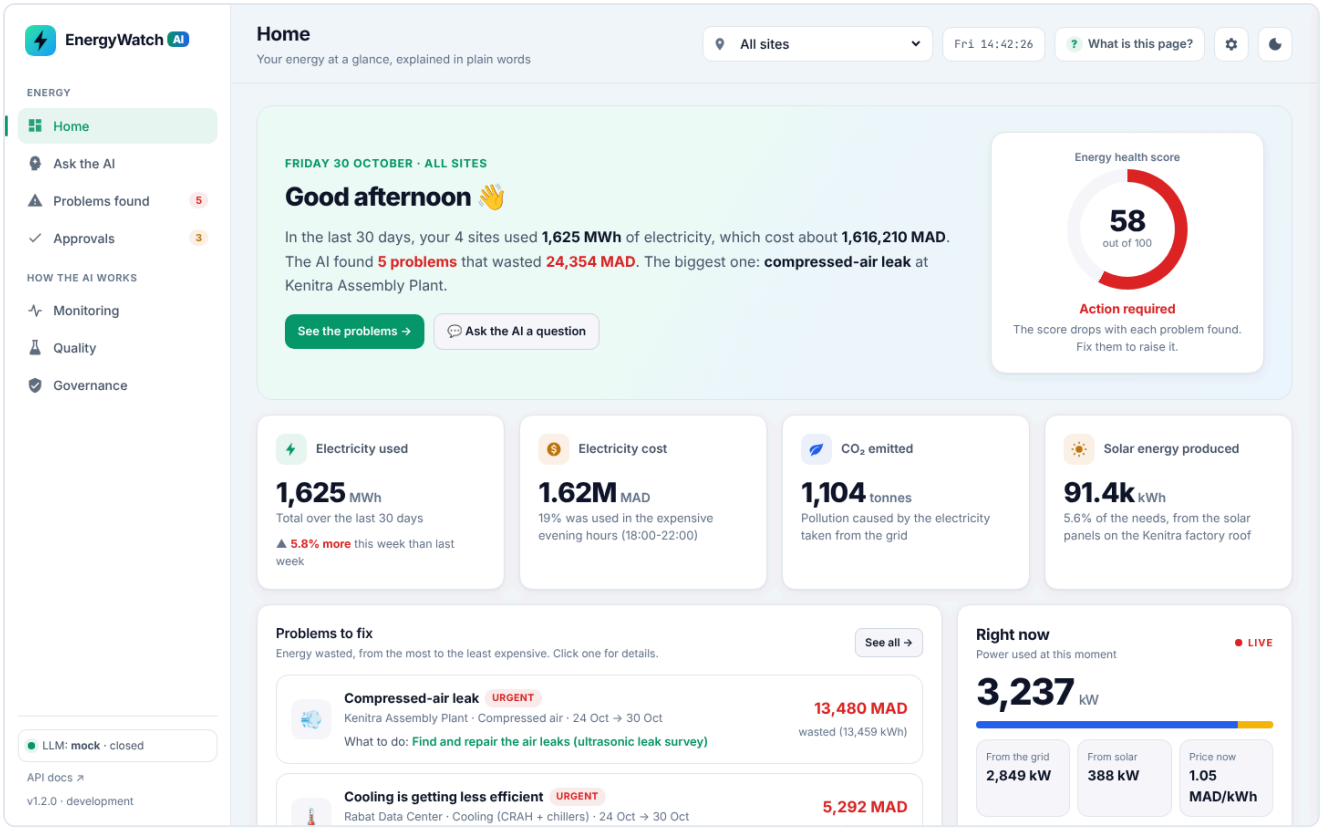
Disjoncteur et mode dégradé

Étape	Comportement
Disjoncteur ouvert ?	Erreur immédiate `LLMUnavailable` (aucune attente)
Appel	Injection de chaos éventuelle, puis jusqu'à 3 tentatives avec délai exponentiel
Échec	`record_failure()` ; au 3e échec consécutif le disjoncteur s'ouvre et l'incident RB-001 est créé
Succès	`record_success()` ; s'il était semi-ouvert, il se ferme et RB-001 est résolu
Côté agent	Interception de `LLMUnavailable`, repli déterministe, span marqué `degraded`

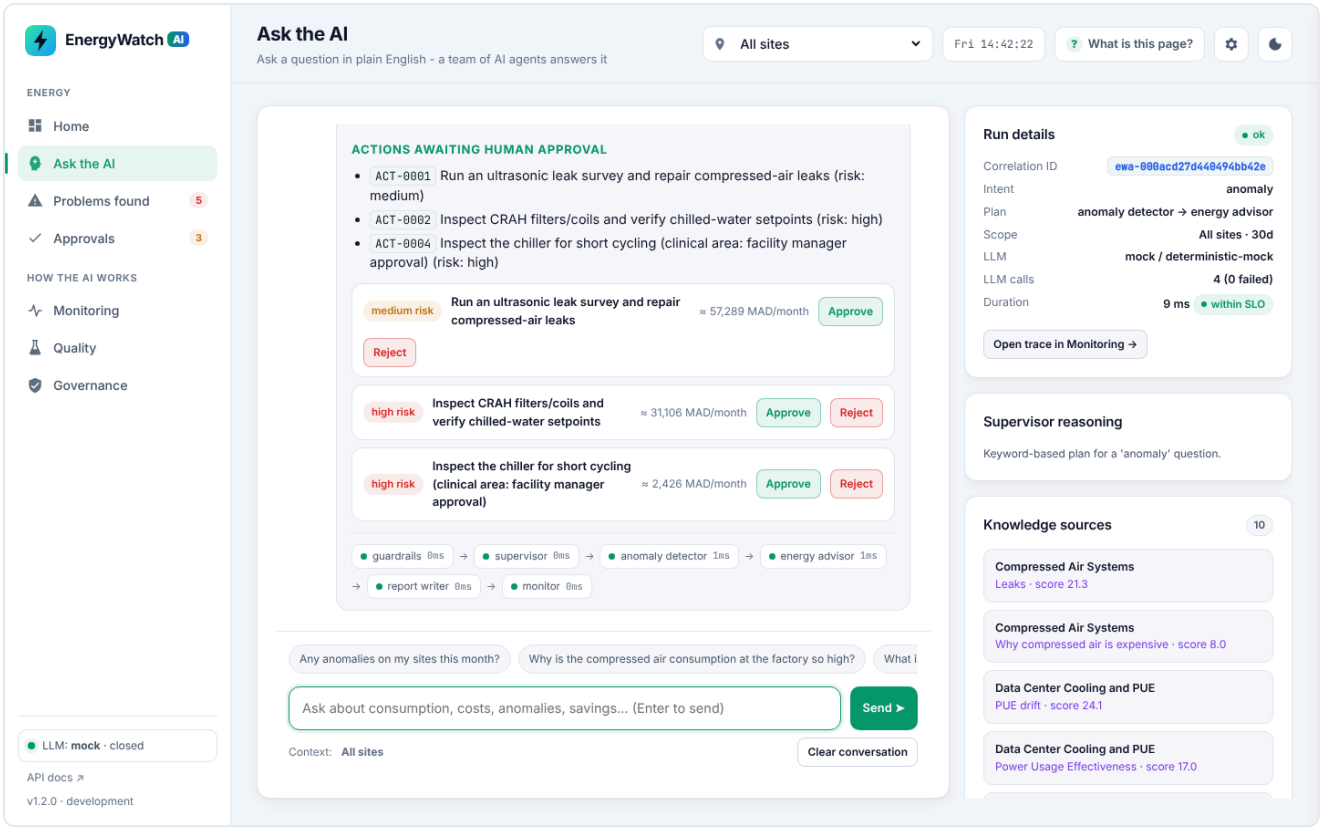
Gestion des incidents

Quatre runbooks sont appliqués automatiquement par l'agent de monitoring : RB-001 (panne du fournisseur LLM), RB-002 (dépassement du SLO de latence), RB-003 (rafale d'injections), RB-004 (panne d'un outil de données). Le détail figure dans le document *Runbook* et l'exercice réel dans le *Rapport d'incident INC-0001*.

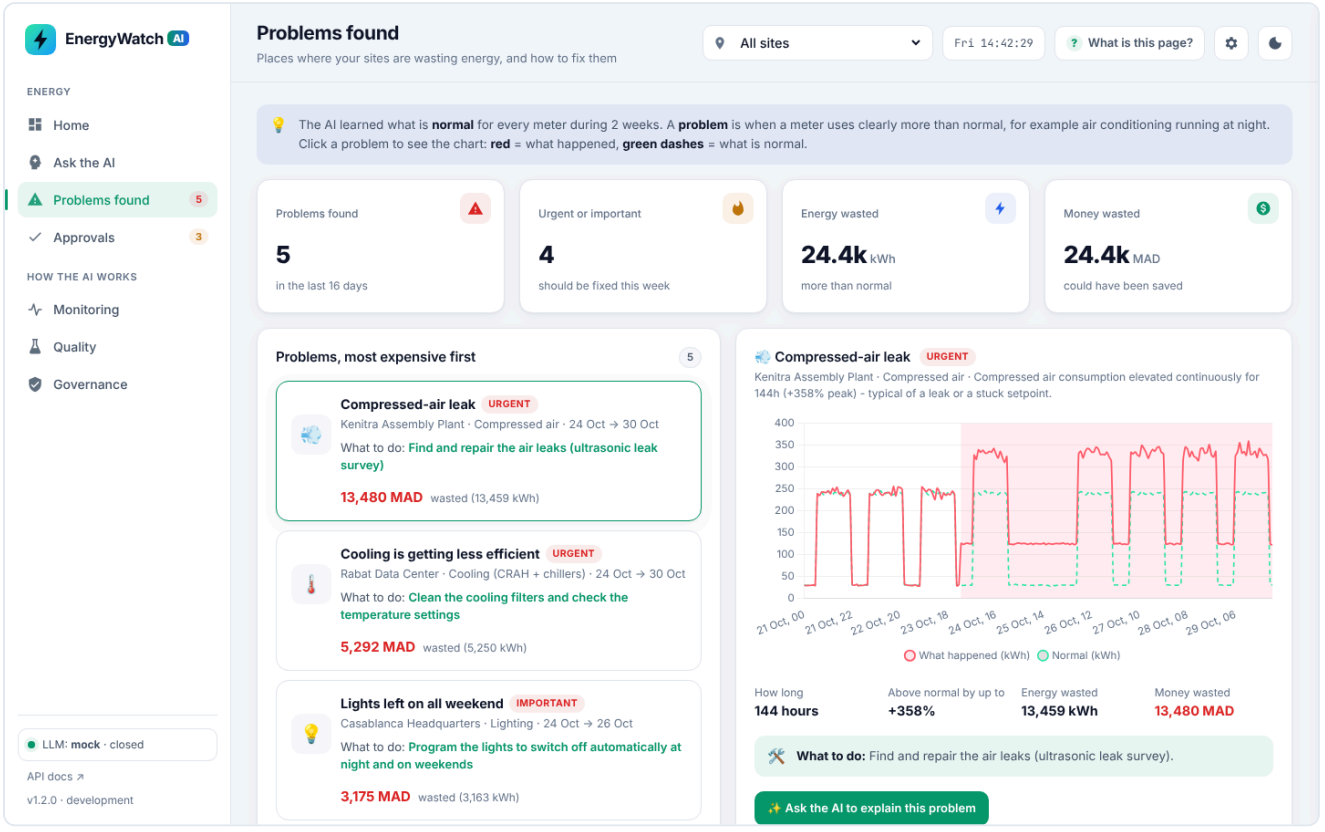
Captures du tableau de bord



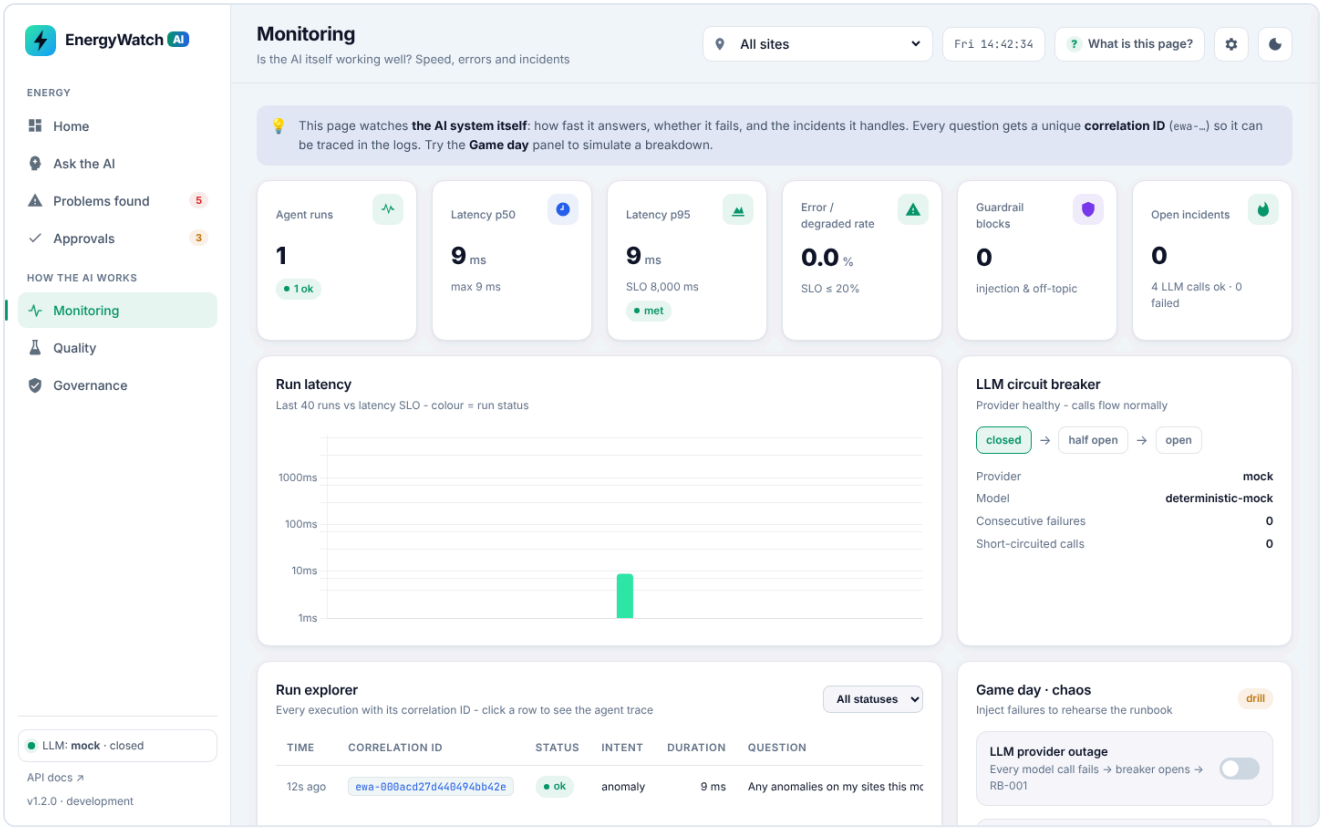
Home : résumé en une phrase, score de santé énergétique, problèmes à corriger



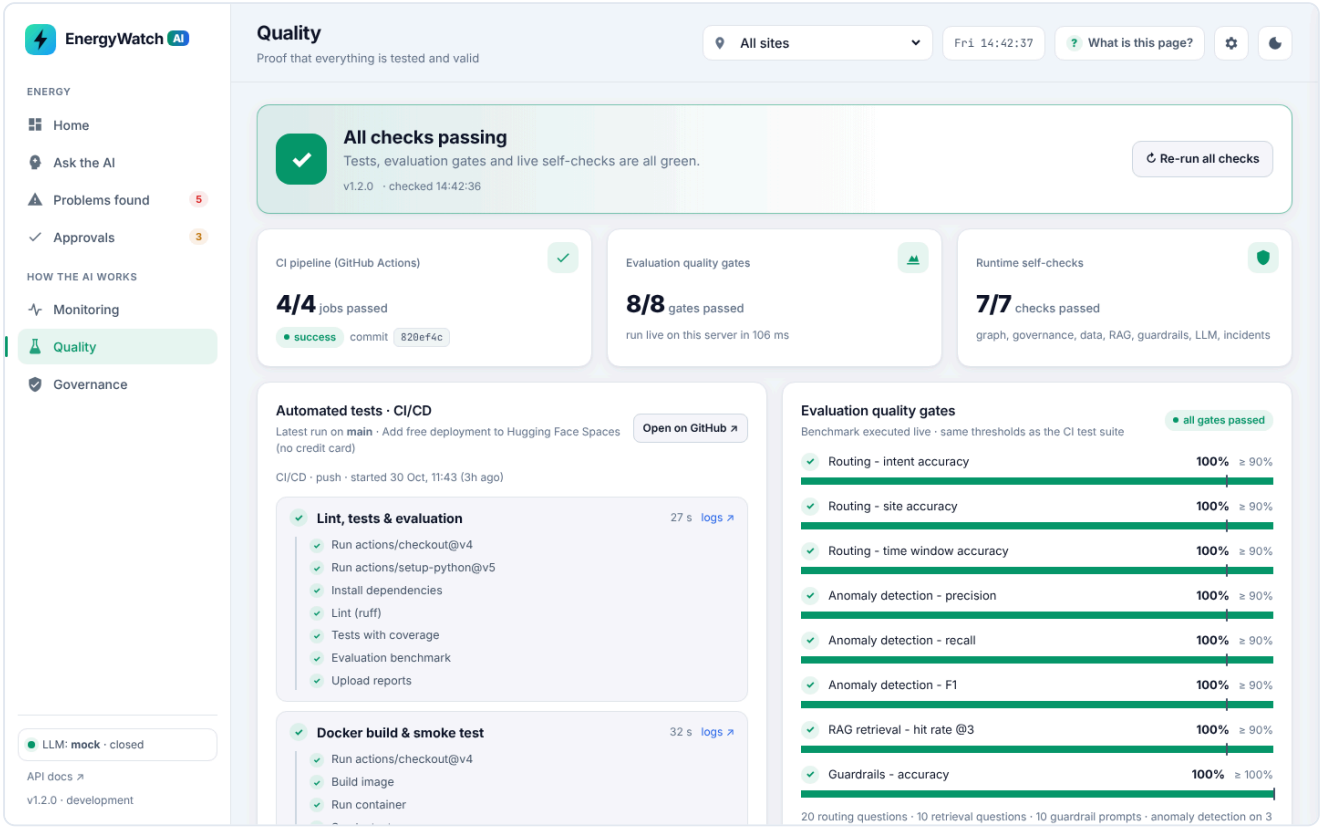
Ask the AI : réponse multi-agents avec plan, sources, validations et trace



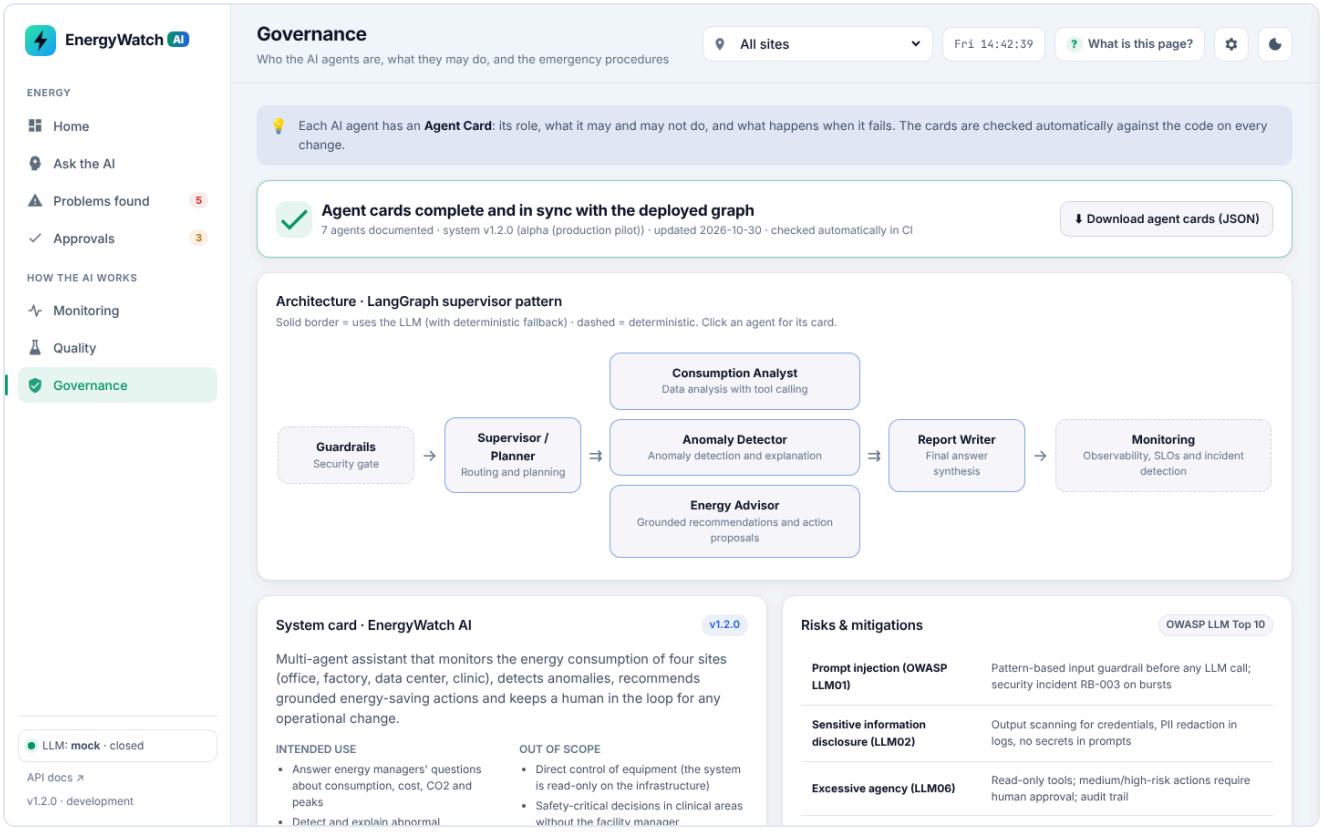
Problems found : consommation réelle comparée à la référence normale



Monitoring : exécutions, SLO de latence, disjoncteur, incidents



Quality : pipeline CI, seuils d'évaluation et vérifications en direct



Governance : Agent Cards, architecture et runbooks

Sécurité et contrôle humain

Risque OWASP	Mesure dans EnergyWatch AI
LLM01 Injection de prompt	Filtre par motifs avant tout appel au LLM ; les requêtes bloquées font 0 appel LLM
LLM02 Divulgation d'informations	Masquage des clés API dans les réponses ; masquage des e-mails et téléphones dans les logs
LLM06 Autonomie excessive	Outils en lecture seule ; actions seulement *proposées* ; validation humaine selon le risque
LLM09 Désinformation	Chiffres calculés par des outils déterministes ; sources citées
LLM10 Consommation non maîtrisée	1 000 caractères maximum ; 30 requêtes / minute / client ; délais d'expiration

Contrôle humain. La politique d'autonomie est la suivante : risque faible (par exemple programmer l'extinction de l'éclairage) = approbation automatique ; risque moyen ou élevé (réparation d'air comprimé, refroidissement du datacenter, climatisation de la clinique) = en attente d'une personne. Les décisions (Approuver / Rejeter, auteur, commentaire, correlation ID) sont inscrites dans une piste d'audit en ajout seul. Les actions sensibles de l'API (approbations, chaos, résolution d'incident) exigent l'en-tête `X-API-Key` quand `ADMIN_API_KEY` est défini.

Tests et évaluation

Suite de tests

70 tests automatisés, exécutés en environ 5 secondes, sans clé API, avec **95 % de couverture** :

Domaine	Ce qui est vérifié
Données	Jeu reproductible, cohérence des totaux, 5/5 anomalies sur plusieurs calendriers
Sécurité	Injections bloquées, hors sujet refusé, masquage des secrets, limitation du débit
RAG	Découpage avec chevauchement, bon document retrouvé
Routage	Intention, site, période, assainissement de la sortie du LLM
Graphe (bout en bout)	Chaque type de question, correlation ID sur chaque span, mode dégradé, panne d'outil
Incidents	Cycle complet RB-001 (ouverture → rétablissement → résolution avec MTTR), RB-002, RB-003, RB-004
API	Santé, en-têtes, validation humaine, protection admin, métriques Prometheus
Gouvernance	Agent Cards complètes et synchronisées avec le graphe
Chemin LLM réel	Modèle simulé qui exerce la sortie structurée, l'appel d'outils et la rédaction

Résultats de l'évaluation

Composant	Indicateur	Résultat	Seuil
Routage du superviseur	Précision de l'intention (20 questions)	**100 %**	≥ 90 %
	Précision du site / de la période	**100 % / 100 %**	≥ 90 %
Détecteur d'anomalies	Précision / rappel / F1	**1,00 / 1,00 / 1,00**	F1 ≥ 0,9
	Robustesse : 30 graines × 7 calendriers	**0 écart sur 210**	testé
Recherche RAG	Taux de succès dans le top 3 (10 questions)	**100 %**	≥ 90 %
Guardrails	Exactitude (10 requêtes)	**100 %**	= 100 %

Lecture honnête des résultats : les jeux de questions sont petits et écrits par l'équipe ; 100 % montre que le système se comporte comme prévu, pas qu'il généralise parfaitement. Les données sont simulées. Avec un vrai LLM, la formulation varie ; on évalue donc la structure (plan, outils appelés, sources présentes, chiffres issus des outils).

Intégration continue (GitHub Actions)

À chaque modification du code : **lint (ruff) → 70 tests + seuil de couverture de 85 % → évaluation → construction de l'image Docker → démarrage du conteneur et smoke test (9 vérifications) → déploiement**. Le dernier résultat est affiché en direct sur la page **Quality** du tableau de bord.

Déploiement

Conteneurisation

- **Dockerfile multi-étapes** : les dépendances sont installées dans une étape de construction ; l'image finale ne contient que l'environnement Python et le dossier `app/`.
- **Utilisateur non-root** (`energywatch`, uid 10001) et **HEALTHCHECK** sur `/health`.
- Respect de la variable `$PORT` (8000 par défaut) ; un seul processus, car la télémétrie est en mémoire.
- Lancement : `docker build -t energywatch-ai .` puis `docker run -p 8000:8000 energywatch-ai`.

Mise en ligne avec Cloudflare Tunnel

L'application est publiée depuis un PC avec **Cloudflare Tunnel**, gratuit et sans compte :

1. Double-cliquer sur `start-online.bat` dans le dossier du projet.
2. Au premier lancement, le script installe l'environnement Python et télécharge Cloudflare Tunnel.
3. Il démarre le serveur, puis affiche une adresse publique du type `https://….trycloudflare.com`.
4. Le lien reste actif tant que le PC et les deux fenêtres restent ouverts.

L'API est documentée automatiquement sur `/docs` (Swagger) à la même adresse.

Configuration

Variable	Défaut	Rôle
`LLM_PROVIDER`	`auto`	`auto`, `groq`, `gemini`, `openai`, `anthropic` ou `mock`
`GROQ_API_KEY`	-	Clé du fournisseur LLM (Groq, gratuit)
`ADMIN_API_KEY`	vide	Protège approbations, chaos et résolution d'incidents
`SLO_LATENCY_MS`	`8000`	SLO de latence utilisé par l'agent de monitoring
`CIRCUIT_BREAKER_THRESHOLD`	`3`	Échecs consécutifs avant ouverture du disjoncteur
`CIRCUIT_BREAKER_RESET_S`	`30`	Délai avant le test en semi-ouvert
`RATE_LIMIT_PER_MINUTE`	`30`	Limite par client sur `/api/agent/ask`

Principaux endpoints de l'API

Méthode	Endpoint	Rôle
`POST`	`/api/agent/ask`	Exécuter le graphe multi-agents sur une question
`GET`	`/api/runs/{correlation_id}`	Trace complète d'une exécution
`GET`	`/api/monitoring/summary`	Latence, taux d'erreur, statistiques par agent, disjoncteur

Méthode	Endpoint	Rôle
`GET`	`/metrics`	Métriques Prometheus
`GET/POST`	`/api/actions`, `/api/actions/{id}/approve`	Validation humaine
`GET/POST`	`/api/incidents`, `/api/chaos`	Incidents et exercices de panne
`GET`	`/api/agent-card`, `/api/quality`	Gouvernance et preuves de qualité
`GET`	`/health`, `/ready`	Sondes de disponibilité

Correspondance avec le cours

Semaine	Notion	Dans EnergyWatch AI
S1	Boucle agentique, taxonomie des modèles	Graphe planifier → agir (outils) → observer → répondre ; passerelle multi-fournisseurs
	Appel d'outils	`bind_tools` dans l'analyste ; arguments validés dans `app/agents/tools.py`
	Prompts stables, sorties structurées, routage	Prompts versionnés ; schéma `RoutingDecision` ; assainissement
	Mémoire et état	`EnergyState` avec réducteurs ; registre des exécutions
S2	RAG fondamental et avancé	7 guides, découpage par section avec chevauchement, BM25, expansion de requête, citations
	Extraction structurée	Mesures horaires brutes → événements `AnomalyEvent` structurés
	Sécurité des agents LLM (OWASP)	Guardrails d'entrée et de sortie, moindre privilège, limitation du débit
S3	Planification	Modèle superviseur / planificateur + répartiteur
	Ingénierie de la fiabilité, résilience	Délais, nouvelles tentatives, disjoncteur, replis, mode dégradé, chaos
	Évaluation et benchmarking	Jeux de référence, précision / rappel, top 3, seuils en CI
S4	Architectures multi-agents, framework	7 agents dans un `StateGraph` LangGraph
	Agents métier, human-in-the-loop, audit	Politique par niveau de risque, Approuver / Rejeter, piste d'audit
	Agents multimodaux	Hors périmètre de cette version (voir perspectives)
S5	Observabilité	Correlation ID, logs JSON, spans, Prometheus, page Monitoring
	Tests et CI/CD agentique	70 tests, GitHub Actions, smoke tests
	Gouvernance, Agent Cards, versionnement	Fiches validées en CI, CHANGELOG
	Déploiement : conteneur et cloud	Dockerfile, Cloudflare Tunnel

Limites et perspectives

Limites actuelles

- Données **simulées** : de vrais compteurs auront plus de bruit, de trous et de changements d'horaire légitimes.
- **Un seul site par question** ; questions en **anglais** uniquement.
- Télémétrie (exécutions, incidents, décisions) **en mémoire** : perdue au redémarrage, les logs restent.
- Mise en ligne par tunnel : le lien ne fonctionne que lorsque le PC est allumé.

Prochaines étapes

1. Gérer le français et le darija, et mieux interpréter les dates.
2. Enregistrer durablement les données et les décisions (base de données, OpenTelemetry).
3. Ajouter une authentification avec des rôles (gestionnaire, technicien, administrateur).
4. Ajouter un agent multimodal qui lit les photos de compteurs et de plaques signalétiques.
5. Connecter de vrais compteurs (MQTT ou API du fournisseur) et mesurer les économies réelles.